



4月19日発売です

ビブリーナ2

店舗特典

- ・メロンブックス
- ・ゲーマーズ
- ・とらのあな
- ・アニメイト
- ・COMICZIN

CHAPTER 1: INTRODUCTION AND A TUTORIAL

Greetings, fellow C learner, the fact that you're reading this book means that you either wanna learn C from another C learner in the modern days (writing this book on the 1st October 2026), or you found the aesthetic appealing, and lowkey that's the exact reason why i made it, i was looking for a book with cute anime girls teachings and actual comprehensible up-to-date book that covers the essentials and is fun to learn..., it's also a remake to the infamous "Second edition: The C programming language" by *Brian Kernighan, Dennis M. Ritchie* .

Anyway, here i would be sharing my knowledge regarding C, as i slowly progress too along the way of writing this book, the book will cover a series of concepts, starting from the raw basics, going through variables and constants, arithmetic, control flow, functions, and the rudiments of input and output, and hopefully pointers and more advanced stuff eventually. We might touch more concepts as we progress and learn.

The approach in use:

In this book we'll be doing real time exercises to make ourselves acquainted with the C syntax and to also tickle the compiler and debug errors if we got any, there would be also some theoretical parts, i know shocker, where we understand certain topics and acronyms like "malloc" (memory allocation) and a "buffer overflow", pretty interesting and important stuff to be honest...

And finally, i'm not the all knowing C expert at the moment, just an enthusiastic developer driven by meme culture and a heavy ego to learn it all, so if you seen something weird or you feel wrong feel free to let me know, i'll be tagging my socials on the book's ending...

1.1.1 Getting started: what is C

C is the godfather of all programming languages, and the Godmother of all operating systems, where C was created in 1972 by Dennis Ritchie to rewrite UNIX, it made Linux Kernel, Windows kernel AND MacOS kernel, pretty cool stuff, it also made Cpython, or basically python language, and mysql too , aside from DOOM and the Quake Engine for it, and finally the computer that guided the apollo mission to space, literally C is the shit, you can basically make anything with it.

1.1.2 Getting started: setting up your environment

To set up your development environment, i advice you to install "clang" package short for "c language" and "gcc" the compiler for C

Choose an IDE you're comfortable in, VScode (install the extensions with it gang), ZED editor (made by the Rust folks, they're our lovely neighbours), or you can go with TUI like me, i got neovim...

It doesn't matter what you use as long as you got clang and gcc AND a terminal

So after you got all that, we'll be starting by creating our first C program, create a new file with the extension c, or just copy the commands (for linux)

```
``` mkdir new_C_program
cd new_C_program
touch main.c ```
```

And yeah start coding...

Our first hello world program:

```
...
#include <stdio.h>
int main(){
 printf("hewo meow meow!");
 return 0;
}
...
```

And just like that you made your first program, we'll be explaining the code too,

`#include <stdio.h>` : is just the importing statement for the standard C library you'll need to print and stuff, cuz the program sees everything as files for some reason, so you need it for now.

`int main()` : your main function, what the program sees to compile and run, anything inside it will be executed, and you'll have it in every program

`printf()` : your output goes here

`return 0;` : every function needs a return, to take out what's inside it

NOW compiling the program to see how to run your code:

1. Go to your terminal and type: `gcc main.c`
2. A new file will appear named `a.out`
3. Run it using: `./a.out`
4. You're supposed to see: `hewo meow meow` , or anything you wrote

To compile the code you wrote to any file name you wish do this

1. Terminal again: `gcc main.c -o file_name`
2. You see the name of the program

Now we're done with the cultural norms when learning a programming language

=> your homework: write a program that prints a lot of statements, maybe a kawaii ASCII drawing you do to theme your terminal or something, one thing tho ``print("\n");`` let's you create another line or perhaps let's say equivalent to hitting the return button

## 1.2 The Basics: Over view

Moving to variables, now you see this shit is reaaaally important you see, unlike other high level languages, C does not hold your hand when you for example type `number = "six seven" ` a high level language \*uhm python uhm\* will save the number as a string  
And what is a string you ask?

A string is at its core a series of characters, and what's a character you said?

A character is a type of data with each character the ability to hold 8 bytes of data... now we have to stop and think about what's a byte and why exactly 8?

We all know that our dear lovely machines cant understand human spoken words, but only 1s and 0s, or more precisely, to whether it has electricity(signal) or not

So what the human race did was to give those signals a meaning and shuffle them into symbols, this table will give u ideas

**8-Bit Binary Alphabet Reference (FIXED)**

Char <b>A</b> Dec <b>65</b> 8-Bit Binary <b>01000001</b>	Char <b>B</b> Dec <b>66</b> 8-Bit Binary <b>01000010</b>	Char <b>C</b> Dec <b>67</b> 8-Bit Binary <b>01000011</b>	Char <b>D</b> Dec <b>68</b> 8-Bit Binary <b>01000100</b>	Char <b>E</b> Dec <b>69</b> 8-Bit Binary <b>01000101</b>	Char <b>F</b> Dec <b>70</b> 8-Bit Binary <b>01000110</b>	Char <b>G</b> Dec <b>71</b> 8-Bit Binary <b>01000111</b>	Char <b>H</b> Dec <b>72</b> 8-Bit Binary <b>01001000</b>
Char <b>I</b> Dec <b>73</b> 8-Bit Binary <b>01001001</b>	Char <b>J</b> Dec <b>74</b> 8-Bit Binary <b>01001010</b>	Char <b>K</b> Dec <b>75</b> 8-Bit Binary <b>01001011</b>	Char <b>L</b> Dec <b>76</b> 8-Bit Binary <b>01001100</b>	Char <b>M</b> Dec <b>77</b> 8-Bit Binary <b>01001101</b>	Char <b>N</b> Dec <b>78</b> 8-Bit Binary <b>01001110</b>	Char <b>O</b> Dec <b>79</b> 8-Bit Binary <b>01001111</b>	Char <b>P</b> Dec <b>80</b> 8-Bit Binary <b>01010000</b>
Char <b>Q</b> Dec <b>81</b> 8-Bit Binary <b>01010001</b>	Char <b>R</b> Dec <b>82</b> 8-Bit Binary <b>01010010</b>	Char <b>S</b> Dec <b>83</b> 8-Bit Binary <b>01010011</b>	Char <b>T</b> Dec <b>84</b> 8-Bit Binary <b>01010100</b>	Char <b>U</b> Dec <b>85</b> 8-Bit Binary <b>01010101</b>	Char <b>V</b> Dec <b>86</b> 8-Bit Binary <b>01010110</b>	Char <b>W</b> Dec <b>87</b> 8-Bit Binary <b>01010111</b>	Char <b>X</b> Dec <b>88</b> 8-Bit Binary <b>01011000</b>
Char <b>Y</b> Dec <b>89</b> 8-Bit Binary <b>01011001</b>	Char <b>Z</b> Dec <b>90</b> 8-Bit Binary <b>01011010</b>						

*\*Printable Latin alphabet, uppercase, standard ASCII/ISO-8859-1 encoding. Corrected binary values.\**

It might look overwhelming at first but it's really easy, each character has its unique 8 bit binary, and a decimal number, to make it into perspective

00000000 (binary) = 1 (decimal); 00000001 = 2 ; 00000010 = 3 ; 00000011 = 4

And so on until you reach your 01000001 = 65

So that's how the computer views your text, pretty cool, you can larp your way across the world using this, like this:

01001001 01001100 01001111 01010110 01000101 01011001 01001111 01010101

Your home work is to decrypt this and translate it.

Now that we know what's a byte and what it can do to make our data types, we shall move on to variables.